

```
>>>type(a)
<type 'numpy.ndarray'>
>>>b=n.array(['a','b','c'])
>>>type(b)
<type 'numpy.ndarray'>
```

Unlike the list data structure in Python, ndarray holds elements of the same data type only. A few attributes of the ndarray object are listed below. Examples shown with the description of each attribute refer to the array 'a', whose definition is as follows:

```
>>>a=n.array([[1,2,3],[3,4,5]])
```

ndarray.ndim: This displays the number of dimensions of the array. The array 'a' in the example is two-dimensional, and hence the output is as follows:

```
>>>a.ndim
2
```

ndarray.shape: This displays the dimensions of the array, as shown below:

```
>>>a.shape
(2,3)
```

ndarray.size: This displays the total number of elements in the array, as shown below:

```
>>>a.size
6
```

ndarray.dtype: This displays the data type of elements, depending on the type of data stored in it. Built-in types include int, bool, float, complex, bytes, str, unicode, buffer; all others are referred to as objects. Default dtype of ndarray is float64.

```
>>>a.dtype
dtype('int64')
```

In the case of strings, 'type' will be displayed as dtype('S#'), where # represents the length of the string:

```
>>>c=n.array(['sarah','serin','jaden'])
>>>c.dtype
dtype('S5')
```

ndarray.itemsize: This displays the size of each element in bytes. In the example, 'a' contains integers and the size of the integers is 8 bytes. Hence, the output is as follows:

```
>>>a.itemsize
8
```

Creating ndarrays

We have already seen in the above examples how ndarrays are created from a Python list using `array()`. A tuple can also be used in place of a list. It is possible to specify explicitly the data type of elements in the array, as shown below:

```
>>>e=n.array([[1,2,3],[3,4,5]],dtype='S2')
>>>e
array([[ '1', '2', '3'],
       [ '3', '4', '5']],
      dtype='|S2')
>>>e.dtype
dtype('S2')
```

There are other ways too for generating arrays. A few of them are listed below (italicised words in the description of each function denote the arguments to the functions).

ones(shape,dtype,order): This returns an array of given dimensions and type filled with 1s. 'Order' in the option set specifies whether to store the data in rows or columns. An example is given below.

```
>>>a=n.ones(3,dtype='S1')
>>>a
array(['1', '1', '1'],
      dtype='|S1')
```

If the specified dtype is Sn, whatever be the value of 'n', the array generated by `ones()` will contain a string of length 1. But, at some later point of time, we will be able to replace '1' with a string of length up to 'n'.

empty(shape,dtype,order): This returns an array of given dimensions and type without initialising the entries. In the code segment given below, the specified dtype is 'S1'. Hence the array 'a' may be modified later to store strings of length 1.

```
>>>a=n.empty(3,dtype='S1')
>>>a
array(['', '', ''],
      dtype='|S1')
```

full(shape,fill_value,dtype,order): This returns an array of given dimensions filled with 'fill_value'. If dtype is not explicitly specified, a float array will be generated with a warning message. A sample statement is given below:

```
>>>a=n.full((3,3),2,dtype=int)
>>>a
array([[2, 2, 2],
       [2, 2, 2],
       [2, 2, 2]])
```

fromstring(string,dtype,count,sep): This returns a 1-D array initialised with 'string'. NumPy takes 'count'